

Intro to Deep Learning

Lecture 09: Embeddings, Metric Learning, and Visual Search

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. What is an embedding?
2. How is retrieval different from classification?
3. Why do we use cosine similarity?
4. What is a visual search pipeline?
5. What is a Siamese network?
6. What is triplet loss trying to do?

Lecture rhythm: embeddings -> cosine search -> retrieval evaluation -> metric learning

Recap From Lecture 08

Attention produces contextual token representations.

Today we return to images, but keep the representation idea:

```
input -> neural network -> useful vector
```

That useful vector is called an embedding.

Embeddings let us compare examples directly.

Part 1

From classification to retrieval

Classification vs Retrieval

Classification asks:

```
which class is this image?
```

Retrieval asks:

```
which stored images are most similar to this image?
```

The output is different:

```
classification -> class logits  
retrieval      -> ranked list of examples
```

Why Retrieval Matters

Many real systems need search, not just labels:

- find similar products
- find similar images
- find duplicate documents
- retrieve related medical cases
- search through videos or audio

The model must organize examples in a useful vector space.

This is where embeddings become central.

Part 2

Embeddings and similarity

Embeddings

An embedding is a vector representation of an input.

For an image:

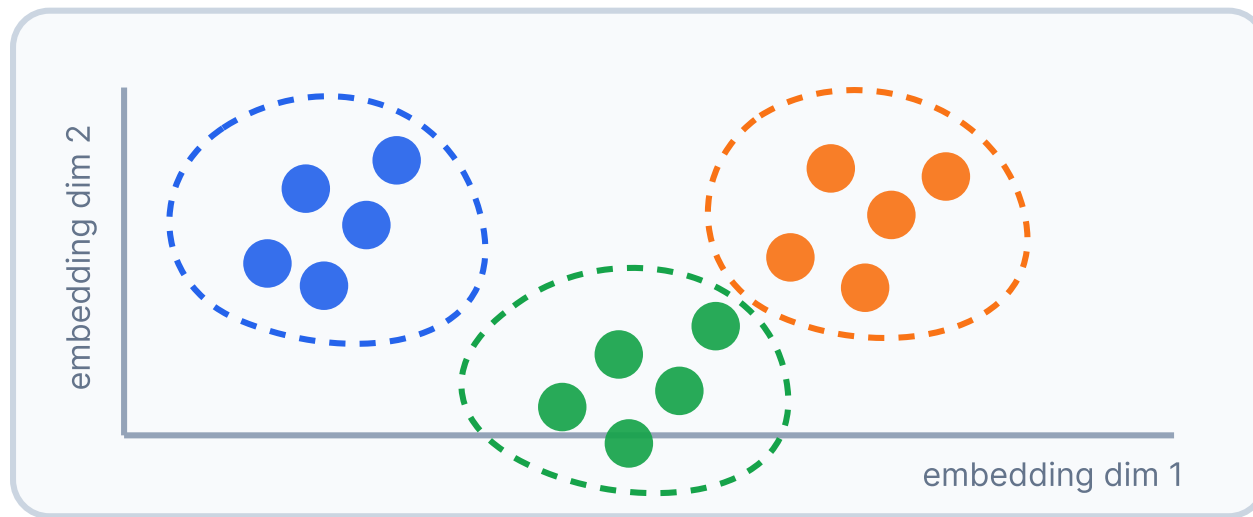
```
image -> encoder -> embedding vector
```

The goal:

```
similar inputs should have nearby embeddings  
different inputs should be farther apart
```


Embedding Space

Embeddings organize examples by similarity



Goal

same meaning
nearby vectors

different meaning
farther apart

Classifier Output vs Embedding

Classifier output:

```
logits for fixed classes
```

Embedding:

```
general-purpose feature vector
```

Example:

```
ResNet18 classifier output: [B, 1000]  
ResNet18 feature embedding: [B, 512]
```

For retrieval, we usually want the embedding.

Reusing Pretrained CNNs

From Lecture 07:

```
image -> pretrained backbone -> feature vector -> classifier head
```

For visual search:

```
image -> pretrained backbone -> feature vector
```

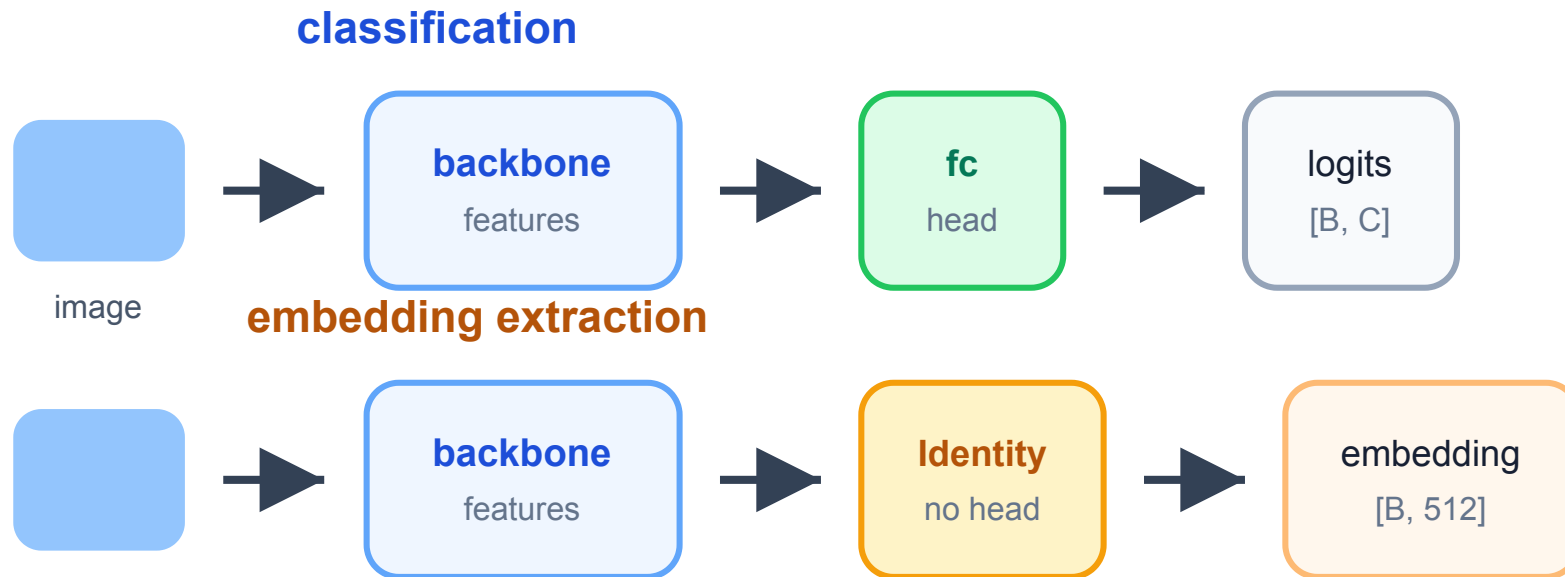
We remove the classifier head.

In Seminar 09:

```
model.fc = nn.Identity()
```

Remove The Classifier Head

For retrieval, keep the backbone and remove the classifier head



Gallery And Query

Visual search uses two sets.

Gallery:

```
stored examples we search through
```

Query:

```
new example we search with
```

In Seminar 09:

```
gallery: CIFAR-10 subset  
query:   smaller CIFAR-10 subset
```

Part 3

Visual search pipeline

Visual Search Pipeline

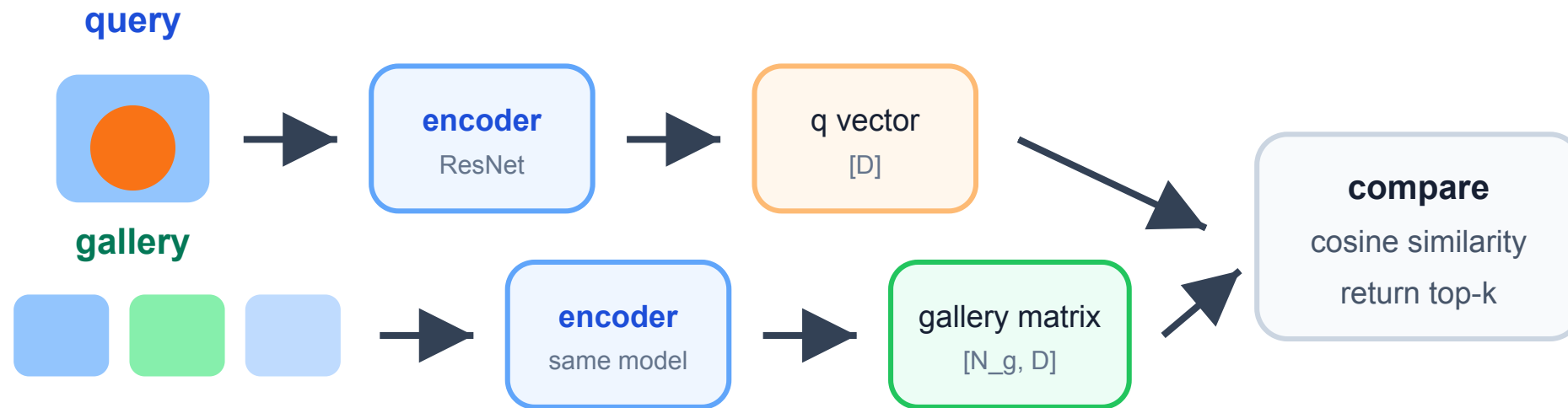
High-level pipeline:

1. encode gallery images
2. store gallery embeddings
3. encode query image
4. compare query embedding to gallery embeddings
5. return top-k most similar images

This is the core practical system.

Retrieval Pipeline

Visual search compares a query vector to gallery vectors



Similarity Measures

To retrieve nearest examples, we need a similarity or distance.

Common choices:

- dot product
- Euclidean distance
- cosine similarity

Similarity:

higher means more similar

Distance:

lower means more similar

Dot Product

Dot product:

$$a \cdot b = \sum_i a_i b_i$$

Large dot product can mean:

similar direction
large vector lengths
or both

So dot product is sensitive to vector magnitude.

This is sometimes good, sometimes inconvenient.

Euclidean Distance

Euclidean distance:

$$d(a, b) = ||a - b||_2$$

It measures direct distance in vector space.

Interpretation:

small distance -> nearby embeddings
large distance -> far embeddings

Triplet loss often uses this kind of distance.

Cosine Similarity

Cosine similarity compares vector direction:

$$\cos(a, b) = \frac{a \cdot b}{||a|| ||b||}$$

Range:

-1 to 1

For retrieval:

higher cosine similarity -> more similar direction

L2 Normalization

If embeddings are L2-normalized:

$$\begin{aligned} ||a|| &= 1 \\ ||b|| &= 1 \end{aligned}$$

Then:

$$\cos(a, b) = a \text{ dot } b$$

This makes similarity computation simple:

$$\text{sims} = \text{query_emb} @ \text{gallery_emb.T}$$

Seminar 09 normalizes embeddings before search.

Mini-Check: Magnitude Trap

Retrieval results are dominated by a few gallery images.

They appear often for many unrelated queries.

You compute similarity with:

```
sims = query_emb @ gallery_emb.T
```

What would you check before changing the model architecture?

Answer

Check whether embeddings were L2-normalized before dot-product search.

Without normalization, dot product is affected by vector magnitude.

Large-norm gallery embeddings can appear similar to many unrelated queries.

Fix:

```
query_emb = F.normalize(query_emb, dim=1)
gallery_emb = F.normalize(gallery_emb, dim=1)
sims = query_emb @ gallery_emb.T
```

Similarity Matrix

With multiple queries and gallery items:

```
query_emb: [Nq, D]  
gallery_emb: [Ng, D]
```

Similarity matrix:

```
sims = query_emb @ gallery_emb.T  
sims.shape = [Nq, Ng]
```

Each row ranks gallery items for one query.

Top-K Retrieval

For one query:

```
compute similarity to all gallery examples  
sort by similarity  
take top k
```

The result is a ranked list:

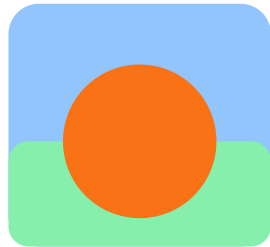
```
most similar -> less similar
```

In Seminar 09, you will visualize the query and its nearest neighbors.

Query And Top-K Results

Retrieval output is a ranked list of examples

query

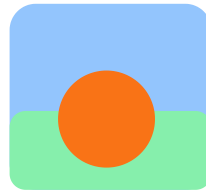


orange object

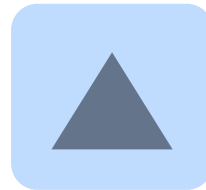
top-5 gallery results



0.93



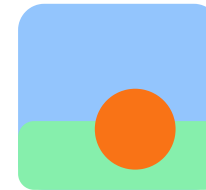
0.88



0.71



0.66



0.63

Inspect both the scores and the images.

Retrieval Is Not Always Perfect

A model can retrieve visually similar but semantically wrong examples.

Example:

```
similar color/background  
different class
```

This is not just a bug.

It reveals what the embedding space considers similar.

Qualitative inspection is part of retrieval evaluation.

Part 4

Evaluating retrieval

Same-Class vs Different-Class Similarity

A useful sanity check:

```
same-class pairs should usually have higher similarity  
than different-class pairs
```

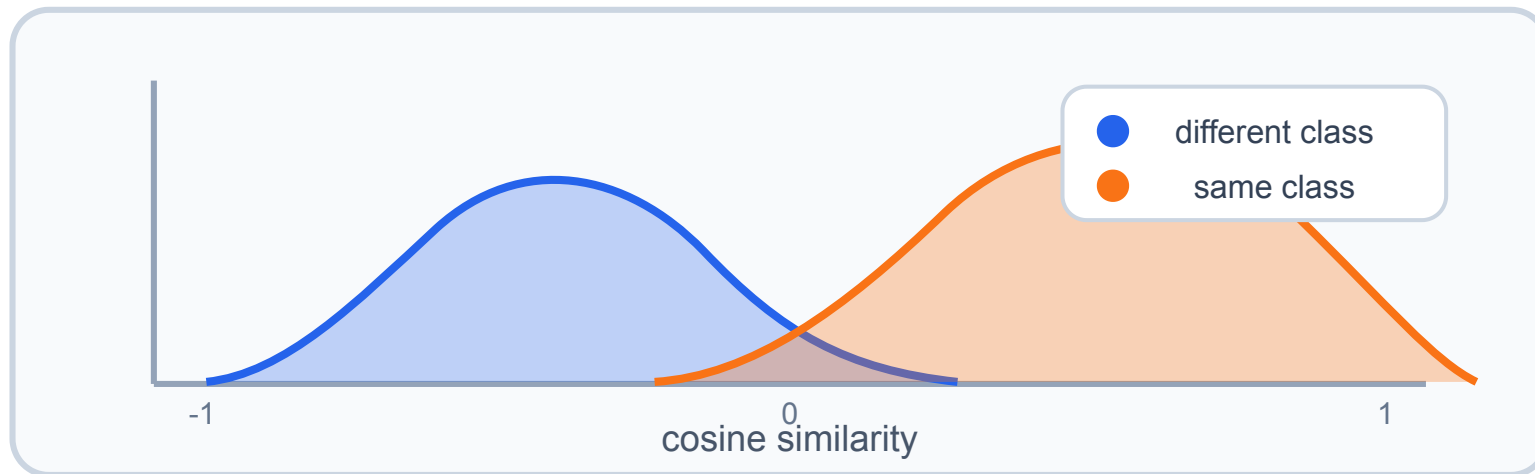
This will not be perfect.

But the distributions should ideally separate somewhat.

If they overlap completely, embeddings may not be useful for the task.

Similarity Distributions

Useful embeddings separate same-class and different-class similarities



Evaluation: Recall@K

For retrieval, accuracy is not the main metric.

Recall@K asks:

is a correct/relevant item present in the top K results?

Example:

Recall@5 = fraction of queries with at least one relevant item in top 5

For CIFAR-10, relevance can mean:

same class label

Part 5

Metric learning

Metric Learning

Metric learning trains embeddings so distances become meaningful.

Goal:

```
same-class examples closer  
different-class examples farther apart
```

Instead of directly predicting class logits, the model learns a space for comparison.

This is useful when retrieval is the final task.

Siamese Networks

A Siamese network compares two inputs with a shared encoder.

```
image A -> encoder -> embedding A  
image B -> encoder -> embedding B
```

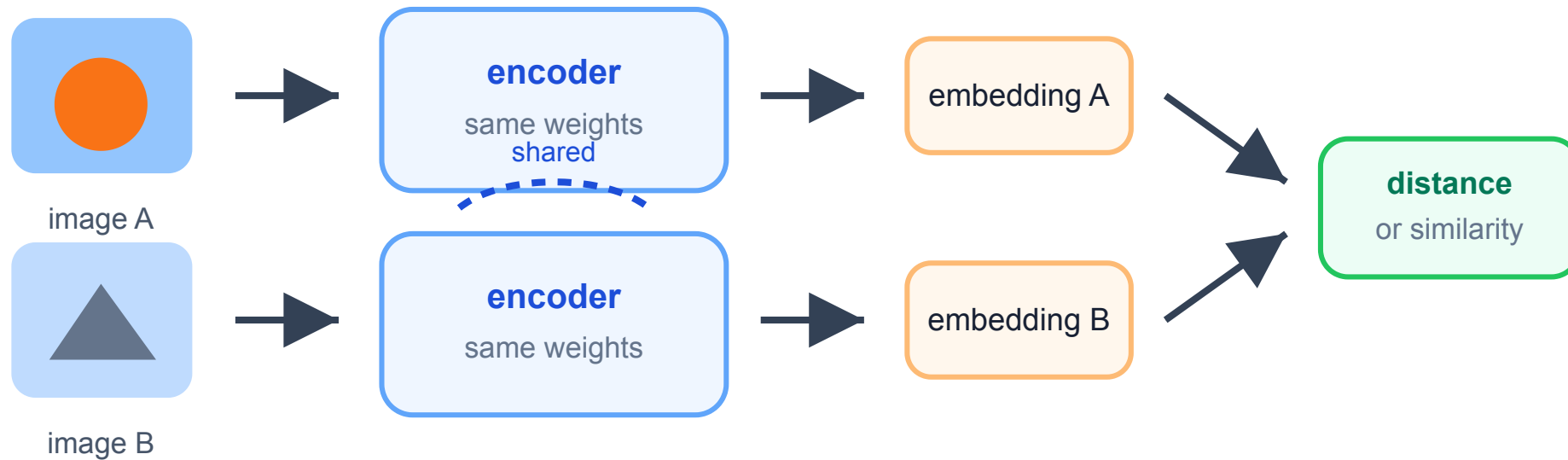
Shared encoder means:

```
same weights process both inputs
```

Then we compare the two embeddings.

Siamese Network

A Siamese network compares two examples with shared weights



Positive And Negative Pairs

Positive pair:

two examples that should be similar

Negative pair:

two examples that should be different

For CIFAR-10:

positive: same class
negative: different class

Pair-based learning uses these labels to shape the embedding space.

Contrastive Learning Intuition

Contrastive objectives usually do two things:

```
pull positives together  
push negatives apart
```

The exact loss can vary.

The core intuition is enough for now:

```
learn embeddings by comparing examples
```

This idea appears in many modern systems.

Triplet Loss

Triplet loss uses three examples:

```
anchor  
positive: same class as anchor  
negative: different class
```

Goal:

```
anchor should be closer to positive  
than to negative
```

Not just slightly closer: closer by a margin.

Triplet Loss Formula

One common version:

$$L = \max (0, d(a, p) - d(a, n) + m)$$

where:

a = anchor, p = positive, n = negative, m = margin

Triplet Loss Intuition

Loss is zero when:

$$d(a, p) + m < d(a, n)$$

Meaning:

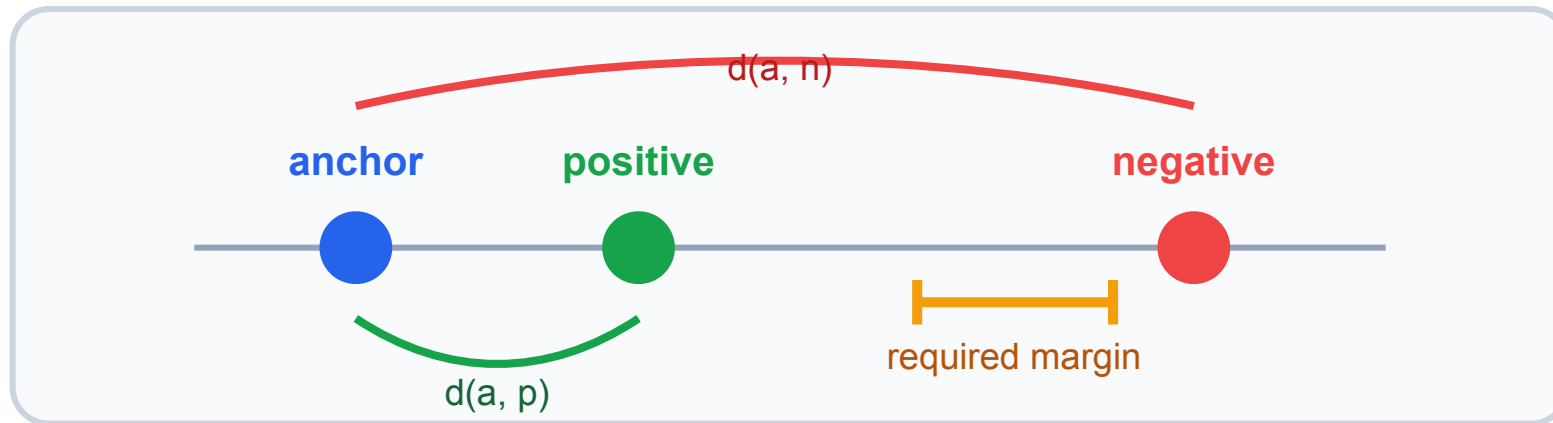
positive is safely closer than negative

If the negative is too close, the loss pushes embeddings to fix it.

This is enough intuition for our course.

Triplet Margin

Triplet loss wants the negative farther away by a margin



Want: $d(a, n) > d(a, p) + \text{margin}$

Practical Limits

Simple retrieval has limits:

- embeddings may capture background more than object identity
- top-k results can be visually plausible but semantically wrong
- class labels may be too coarse
- small gallery results can be unstable
- pretrained features may not match the target domain

Inspect examples, not only metrics.

Common Implementation Mistakes

Mistake 1:

```
using classifier logits as retrieval embeddings
```

Mistake 2:

```
forgetting L2 normalization before cosine search
```

Mistake 3:

```
mixing query and gallery dimensions
```

Mistake 4:

```
forgetting model.eval() during embedding extraction
```

Debugging Checklist

Before retrieval, check:

- embedder outputs $[B, D]$
- gallery embeddings have shape $[N_g, D]$
- query embeddings have shape $[N_q, D]$
- embeddings are L2-normalized
- similarity matrix has shape $[N_q, N_g]$
- top-k indices refer to gallery examples
- visualized labels match retrieved images

Seminar Bridge

In Seminar 09, you will:

1. load pretrained ResNet18
2. remove the classifier head
3. extract normalized image embeddings
4. compute cosine similarity
5. retrieve top-k gallery images
6. compare same-class and different-class similarities
7. optionally explore Siamese pairs and triplet loss

Main goal:

build a small visual search system

Takeaways

Embeddings turn inputs into comparable vectors.

Key ideas:

- retrieval returns examples, not class labels
- cosine similarity works well with normalized embeddings
- visual search is query embedding vs gallery embeddings
- metric learning shapes the embedding space
- triplet loss pulls positives closer than negatives by a margin

Next lecture:

object detection and YOLO